

Lab 1

HLD vs LLD Walkthrough — ANSWER KEY

Day 1 | System Design Training Program | Individual Worksheet | Case Study: RetailOrderHub

Prepared for trainer use — reference answers and rationale for facilitating discussion.

Objective

Practice distinguishing High-Level Design (HLD) from Low-Level Design (LLD) by classifying fifteen sample diagrams — including RetailOrderHub's own current architecture — and justifying each answer against the 5-point framework below.

Quick Reference — The 5-Point Framework

	HLD	LLD
Focus	Overall system architecture	Component-level details
Audience	Architects, stakeholders	Developers, technical teams
Abstraction	Abstract and conceptual	Detailed and concrete
Output	Block diagrams, system overview	Class diagrams, pseudo-code
Purpose	Define system structure	Guide implementation

Diagram 1 — RetailOrderHub: Current Architecture

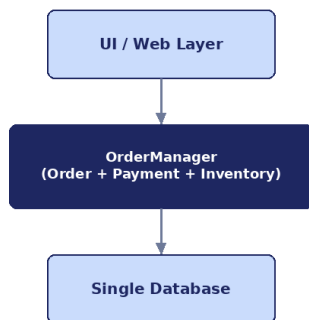


Diagram 1

My classification: ☒ HLD ☐ LLD

Justification (which of the 5 framework points led you here?)

HLD — the diagram shows only three coarse-grained blocks (UI/Web Layer, OrderManager, Single Database) with no internal detail. That matches Focus (overall system architecture) and Abstraction (abstract and conceptual); it's the kind of block diagram an architect or stakeholder would use to understand the system at a glance, not something a developer could implement directly from.

Notes

- OrderManager bundles Order + Payment + Inventory responsibilities into a single monolithic component — a design smell (SRP violation) that stays hidden at this abstraction level.
- This is the 'current state' the course uses as a starting point before decomposing into services (compare with Diagram 3).

Reflection

What would this system's LLD need to show that this HLD doesn't? Name at least two things.

- 1) The internal class structure once OrderManager is decomposed — attributes and method signatures like those in Diagram 2 (and later Diagram 8's Strategy-pattern breakdown of PaymentGateway).
- 2) The sequence of calls between components for a real use case, e.g. placing an order (submitOrder → createOrder → charge → return confirmation), as shown in Diagram 4, including error/failure handling.
- 3) The database design — actual tables, keys, and relationships — instead of a single 'Single Database' box.

Diagram 2

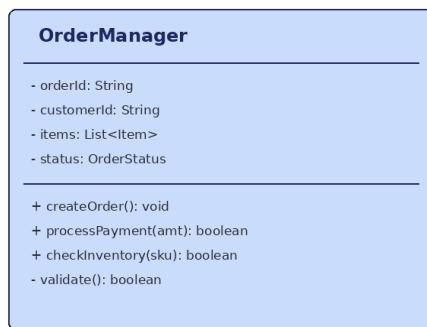


Diagram 2

My classification: ☐ HLD ☒ LLD

Justification (which of the 5 framework points led you here?)

LLD — this is a class diagram exposing internal attributes (orderId, customerId, items, status) and method signatures with visibility modifiers. That's component-level detail meant for developers (Audience), expressed as a class diagram (Output) — both point squarely to LLD.

Notes

- This is what's 'inside the box' of the OrderManager block from Diagram 1.
- Note the private `validate()` method sitting alongside public order/payment/inventory operations — a preview of the SRP issue the Day 2 SOLID lab addresses by extracting `PaymentService`.

Diagram 3

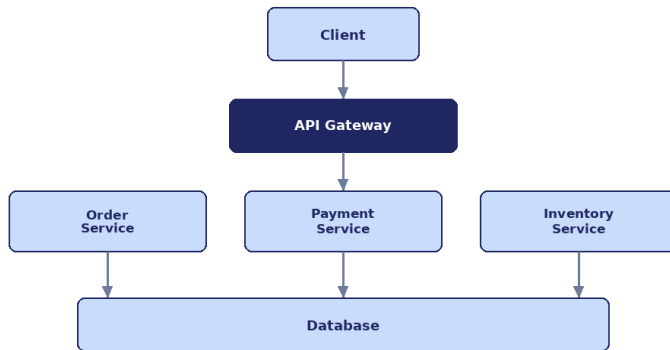


Diagram 3

My classification: ☒ HLD ☐ LLD

Justification (which of the 5 framework points led you here?)

HLD — it's a block diagram of the system's services (API Gateway, Order/Payment/Inventory Service, Database) and how requests flow through them, with no class or method-level detail shown. Same abstraction level as Diagram 1's Focus and Output, just with the monolith decomposed into services.

Notes

- Looks like a possible 'future state' of Diagram 1 after breaking up OrderManager — still HLD because it only shows service boundaries, not what's inside each service.
- Worth flagging in discussion: this is the diagram most likely to spark debate (see Wrap-Up).

Diagram 4

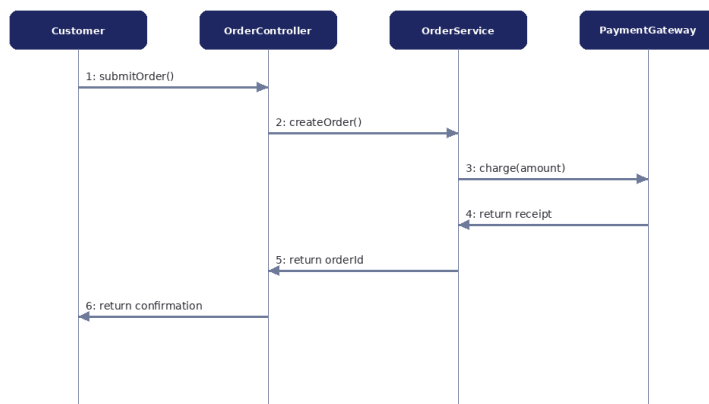


Diagram 4

My classification: ☐ HLD ☒ LLD

Justification (which of the 5 framework points led you here?)

LLD — this sequence diagram pins down the exact order of method calls (submitOrder → createOrder → charge(amount) → return receipt → return orderId → return confirmation) between specific components. That's implementation-guiding detail (Purpose) for developers, not an architectural overview.

Notes

- Sequence diagrams like this are typically produced during detailed design, once an HLD (like Diagram 3) has already established which services exist.
- Useful contrast with Diagram 3: same underlying system, but this view is about behavior/interaction, not structure.

Diagram 5

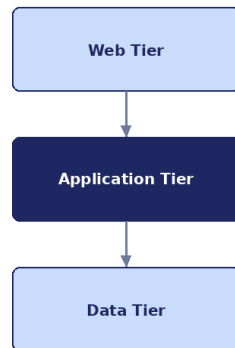


Diagram 5

My classification: ☒ HLD ☐ LLD

Justification (which of the 5 framework points led you here?)

HLD — a classic three-tier (Web / Application / Data) architecture diagram. It communicates overall system structure and separation of concerns at a conceptual level appropriate for architects and stakeholders, with no component or class-level detail.

Notes

- Generic tiered view — could describe almost any web application, including a simplified early view of RetailOrderHub before its services were named.

Diagram 6

```
function calculatePay(employee):  
    hours = getHours(employee)  
    rate = getHourlyRate(employee)  
    pay = hours * rate  
  
    if hours > 40:  
        overtime = (hours - 40) * rate * 1.5  
        pay = pay + overtime  
  
    return pay
```

Diagram 6

My classification: ☐ HLD ☒ LLD

Justification (which of the 5 framework points led you here?)

LLD — pseudo-code is explicitly listed as an LLD Output in the framework. It specifies exact variables, arithmetic, and control flow (the overtime if-branch) needed to implement the function — concrete and developer-facing, not conceptual.

Notes

- Not part of RetailOrderHub directly (a payroll example) — included to test whether the classification skill generalizes beyond the case study.

Diagram 7

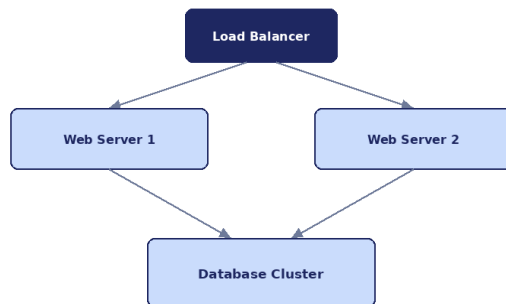


Diagram 7

My classification: ☒ HLD ☐ LLD

Justification (which of the 5 framework points led you here?)

HLD — another block-level diagram, illustrating horizontal scaling and high availability (load balancer, two web servers, database cluster) at the architecture level. It shows what components exist and how they relate, not how any one of them is implemented.

Notes

- Directly relevant to Day 3 (Scalability & Resilience) — this is the level at which you'd reason about load balancing and redundancy before drilling into implementation.

Diagram 8

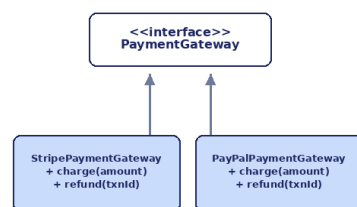


Diagram 8

My classification: ☐ HLD ☒ LLD

Justification (which of the 5 framework points led you here?)

LLD — a UML class diagram showing a specific interface (PaymentGateway) and its concrete implementing classes (StripePaymentGateway, PayPalPaymentGateway) with method signatures. That's exactly the level of

design decision — which pattern, which classes — a developer needs, not something a stakeholder would review.

Notes

- This is the Strategy pattern applied to PaymentGateway — directly relevant to Day 2's OCP lab, where participants extend payment behavior without modifying existing code.

Diagram 9

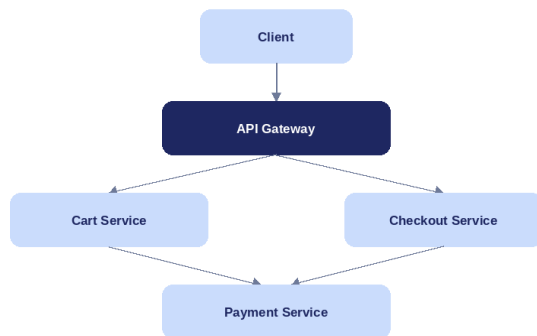


Diagram 9

My classification: ☒ HLD ☐ LLD

Justification (which of the 5 framework points led you here?)

HLD — a block diagram of services (API Gateway, Cart Service, Checkout Service, Payment Service) and how a request flows between them, with no method signatures or internal logic shown. Same Focus and Output as Diagrams 3 and 7 — architecture, not implementation.

Notes

- A good pairing with Diagram 3 — same reasoning applies even though the domain (checkout) is more specific and 'shopping-flavored.'

Diagram 10

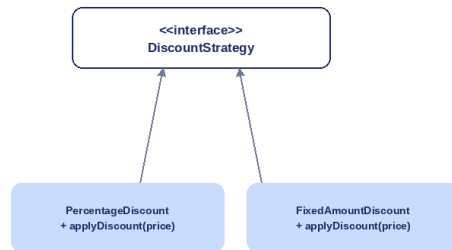


Diagram 10

My classification: ☐ HLD ☒ LLD

Justification (which of the 5 framework points led you here?)

LLD — the same interface-plus-concrete-implementations pattern as Diagram 8, just applied to discounts instead of payments: an interface (`DiscountStrategy`) with a method signature, and two classes implementing it. Component-level detail meant for developers.

Notes

- Another Strategy pattern example — useful for reinforcing that OCP applies far beyond payments; any ‘family of algorithms’ (discounts, shipping costs, tax rules) is a Strategy candidate.

Diagram 11

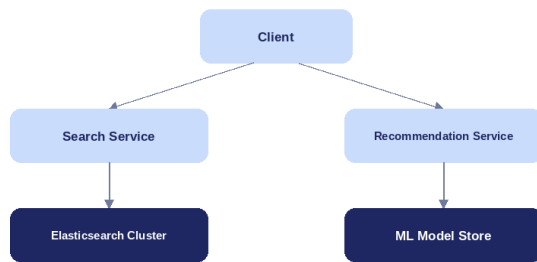


Diagram 11

My classification: ☒ HLD ☐ LLD

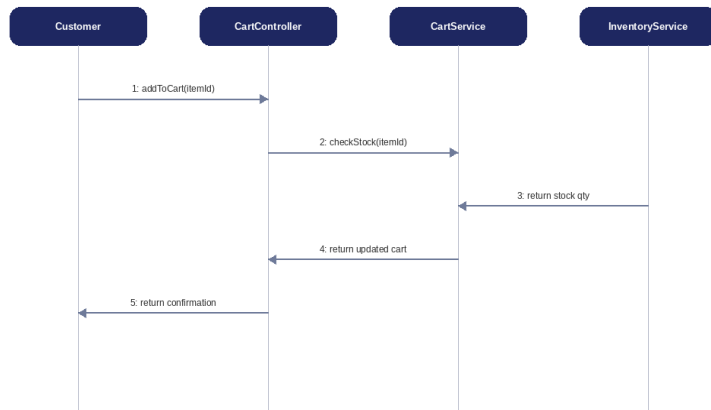
Justification (which of the 5 framework points led you here?)

HLD — still a block diagram of services and the infrastructure they depend on (Elasticsearch, an ML model store), with zero class or method detail. Naming specific technologies doesn't make a diagram LLD — Abstraction and Output are what matter, and both stay high-level here.

Notes

- A good 'trap' diagram — participants sometimes call this LLD because it names real technologies (Elasticsearch, ML model store). Push back: does it show classes, methods, or call order? It doesn't — that's the tell.

Diagram 12



My classification: ☐ HLD ☒ LLD

Justification (which of the 5 framework points led you here?)

LLD — a sequence diagram pinning down the exact call order between named components (CartController → CartService → InventoryService) for one specific use case. Same reasoning as Diagram 4: Purpose is to guide implementation, not communicate architecture.

Notes

- Compare directly with Diagram 4 — both are sequence diagrams, different use cases (checkout vs. add-to-cart), same classification logic.

Diagram 13

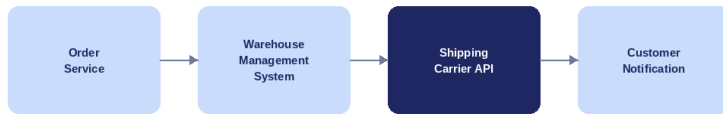


Diagram 13

My classification: ☒ HLD ☐ LLD

Justification (which of the 5 framework points led you here?)

HLD — a horizontal chain of systems (Order Service, Warehouse Management System, Shipping Carrier API, Customer Notification) showing what talks to what, with no internal detail on any one box. Same level as Diagrams 3, 7, and 9.

Notes

- Note the shape: this is a straight chain rather than a tree — a good moment to point out that HLD block diagrams can take many shapes; it's the abstraction level that defines HLD, not the layout.

Diagram 14

```
function applyCoupon(cart, code):  
  
    coupon = lookupCoupon(code)  
  
    if coupon is None or coupon.expired:  
        return cart  
  
    if cart.subtotal < coupon.minPurchase:  
        return cart  
  
    cart.discount = coupon.calculateDiscount(cart.subtotal)  
    cart.total = cart.subtotal - cart.discount  
  
    return cart
```

Diagram 14

My classification: ☐ HLD ☒ LLD

Justification (which of the 5 framework points led you here?)

LLD — pseudo-code, like Diagram 6, is explicitly an LLD Output. It specifies exact variables, comparisons, and control flow (the expiry and minimum-purchase checks) needed to implement the function.

Notes

- Direct parallel to Diagram 6 — useful to ask participants what the two pseudocode diagrams have in common structurally (both are guard-clause-heavy, single-function algorithms).

Diagram 15



Diagram 15

My classification: ☒ HLD ☐ LLD

Justification (which of the 5 framework points led you here?)

HLD — despite the event-driven flavor (an event bus, an async detector job), this is still a block-level diagram of components and the direction data flows between them. No class, method, or message-schema detail is shown.

Notes

- Along with Diagram 11, this is a strong 'could go either way' candidate for the wrap-up discussion — participants sometimes read 'event-driven' as inherently more detailed/technical than a simple three-tier diagram, but the framework's Output and Abstraction columns still say HLD.

Wrap-Up — Discussion Answers

Which diagram was hardest to classify, and why?

Diagram 3 (API Gateway → services → Database) is the one participants most often hesitate on. It names individual services, which can feel like 'detail,' but it still only shows system-level blocks and their relationships — no attributes, methods, or call sequences — so it stays HLD by the framework's Output and Abstraction columns. Diagrams 11 and 15 tend to draw the same hesitation, for the same reason — naming real technologies (Elasticsearch, Kafka) feels like detail, but neither shows a class, method, or message schema.

Did any diagram feel like it could reasonably be argued either way?

Diagram 3 again, and to a lesser extent Diagram 4. Diagram 4's sequence diagram is 'detailed' in the sense of showing real interactions, which can read as HLD-ish system behavior — but because it specifies exact method calls and order between named components, it lands on LLD (Purpose: guide implementation). Diagram 15 is this course's newest example of the same trap in the other direction: its event-driven shape (event bus, async detector) can read as more 'technical' than a plain three-tier diagram, but it never drops below the block level, so it stays HLD.

For RetailOrderHub specifically — what's one design smell you'd expect its LLD to expose that the HLD hides?

The God Object / Single Responsibility violation in OrderManager: Diagram 1's HLD shows one clean, innocuous-looking block, but Diagram 2's class-level view reveals that OrderManager owns order creation, payment processing, and inventory checks all at once — the exact problem the Day 2 SOLID lab refactors away.